1. Objetivos

El objetivo de este trabajo es que el estudiante seleccione un problema computacional cuyo código secuencial sea razonablemente sencillo, y lo analice, optimice y paralelice aplicando las distintas técnicas vistas en la asignatura.

El resultado final debe demostrar la capacidad del estudiante para identificar paralelismo, escoger técnicas adecuadas, implementarlas y evaluar su impacto en el rendimiento.

2. Elección del problema.

El estudiante deberá elegir un problema computacional apropiado. Algunos ejemplos posibles (no exhaustivos):

- Estimación de números matemáticos (π mediante Monte Carlo, *e* mediante series, etc.)
- Algoritmos de cálculo científico (multiplicación de matrices, transformadas, integrales numéricas, ecuaciones diferenciales, etc.)
- Procesamiento de imágenes o señales
- Algoritmos clásicos: búsqueda, ordenación, grafos, n cuerpos, etc.
- Generador de números aleatorios.
- Simulación de cellular automata (por ejemplo, Juego de la Vida)

El estudiante podrá usar código propio o código existente, siempre que indique la referencia y sea capaz de explicar su funcionamiento. El objetivo no es usar un código complejo, sino por el contrario uno lo más simple posible pero en el que el efecto de la ejecución en sistemas paralelos o distribuidos sea relevante.

3. Análisis e implementación de optimizaciones.

Para el problema elegido, se pide:

a) Descripción de la plataforma.

Se debe describir la plataforma y configuración usada para las pruebas, tanto a nivel hardware (procesador, memoria) como software (sistema operativo, compilador, etc.). El detalle de la descripción debe ser suficiente como para que los experimentos puedan ser replicados obteniendo resultados similares. Como plataforma pueden usarse las máquinas de los laboratorios, portátil/equipo propio, plataforma en la nube, etc. En todo caso, debe tener en cuenta que los tiempos de ejecución cuando se usa computación en la nube, máquinas virtuales o WSL (Windows Subsystem for Linux) pueden ser engañosos. Se valorará especialmente la ejecución en el *cluster* del departamento (ver más adelante).

b) Análisis del código secuencial.

Se debe analizar el código secuencial del problema elegido, identificando las dependencias (ej. grafo de flujo de datos) y el paralelismo disponible (ILP, DLP y TLP). Analice los posibles límites teóricos (Ley de Amdahl, modelo de tejado, Intensidad Aritmética, ancho de banda disponible, etc.).

c) Implementación de optimizaciones y paralelizaciones

Se espera que el estudiante aplique una o varias de las técnicas estudiadas en la asignatura (tantas como el problema permita). Por ejemplo:

- ILP: reordenamiento, desenrollado de bucles, *software pipelining*, eliminación de dependencias, predicción de saltos, etc. Estas optimizaciones pueden hacerse "a mano" (en cuyo caso se describirá cómo se ha modificado el código) o bien ser realizadas automáticamente por el compilador (en cuyo caso debe analizarse y explicarse el código ensamblador generado).
- DLP: uso de instrucciones SIMD (debe especificarse qué juego de instrucciones se usa y cómo puede influir en las prestaciones), ejecución en GPU (puede usarse Google Colab o similar si no se dispone de tarjeta gráfica), etc.
- Paralelismo de hilos (TLP): ej. uso de OpenMP para paralelización a nivel de varios cores.
- Paralelismo distribuido: ej. MPI (si el problema lo permite)

No es obligatorio implementar todas; depende del problema seleccionado. Pero se valorará la diversidad y la calidad. Es obligatorio explicar con detalle en qué medida estas optimizaciones mejorarían la ejecución del código.

d) Evaluación Experimental

Se deberá hacer una evaluación cuantitativa, comparando los tiempos de ejecución de la versión secuencial y de la versión optimizada. Dicha evaluación debe incluir lo siguiente:

- Medición del tiempo de ejecución, solo de la parte optimizada. Puede usarse alguna de las funciones de medición de tiempos estudiadas (rdtsc(), omp_get_wtime(), etc.).
- Cálculo de parámetros como: aceleración obtenida, eficiencia, escalabilidad, etc.
- Comparación entre los diferentes enfoques analizados (OpenMP vs SIMD, GPU vs CPU, etc.)
- Discusión de posibles cuellos de botella.

4. Presentación de resultados

Deberá realizarse en grupos de 2 personas, y consistirá en una breve defensa oral que durará aprox. 10 minutos más preguntas (a ambos miembros del grupo). Se usará una breve presentación Power Point o similar que incluirá los contenidos descritos en los apartados anteriores (problema y plataforma elegidos, código/s usado/s, resultados obtenidos, etc.). La puntuación obtenida será individualizada, según la presentación y las respuestas de cada miembro del grupo. Se valorará:

- Una adecuada descripción del problema, plataforma y código elegido, que debe ser muy breve y centrada en el paralelismo potencial.
- Explicación de todas las técnicas aplicadas. Se valorará el uso de las métricas y técnicas estudiadas en la asignatura.
- Resultados experimentales, con gráficas y tablas.
- Discusión y conclusiones.
- La calidad de la presentación y de las respuestas a las preguntas del profesor/a.
- Es importante centrarse en los aspectos más importantes y en las aportaciones propias, no repetir lo que ya se ha visto en la asignatura (que se supone ya estudiado). La presentación solo puede durar 10 minutos, debe aprovecharse el tiempo. Se recomienda encarecidamente ensayar previamente para controlar el tiempo de exposición.

Entrega

Se deberá entregar en un fichero ZIP, identificado con el grupo de prácticas y el número de grupo (Por ejemplo, **Trabajo_L1_G3.zip**) la presentación, carpeta con códigos implementados y hojas de cálculo con los resultados presentados.

Extras de carácter voluntario a valorar

Las siguientes opciones son voluntarias, pero se valorarán especialmente, mejorando la calificación:

- La implementación mediante CUDA
- La combinación de OpenMP y MPI
- La ejecución en clúster:

Se realizará prueba en un clúster real de la parte MPI o/y de OpenMP y MPI.

Para ello, se debe comunicar a la profesora Mª José Morón Fernández mediante email (mjmoron@us.es), para poder asignarle usuario y contraseña de acceso.

Previamente a la ejecución en el cluster, es altamente recomendable tener depurados y probados los ejercicios que se vayan a ejecutar en el mismo.

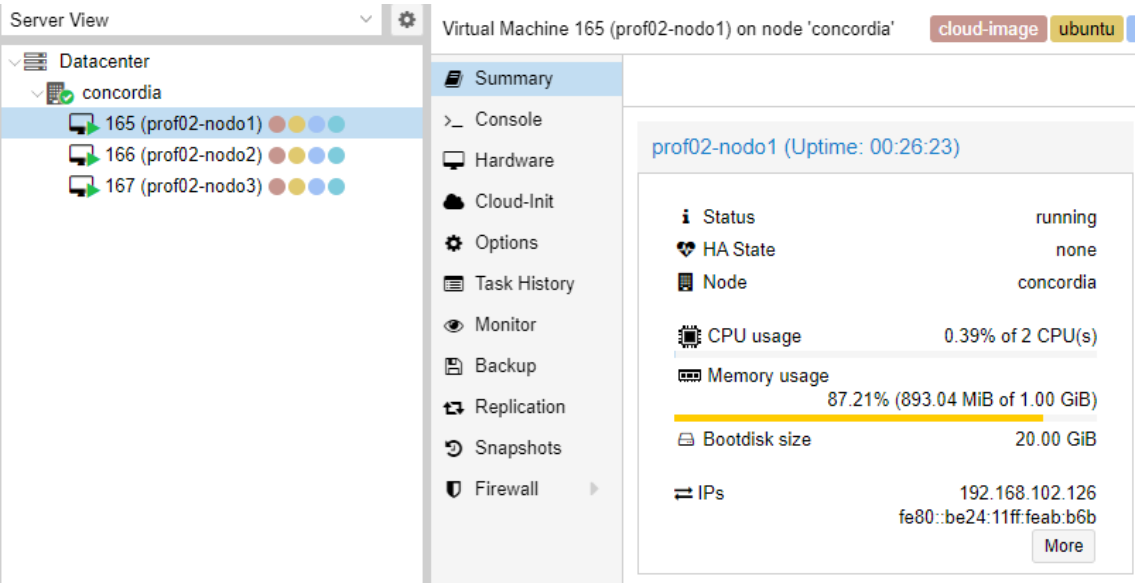
Para la ejecución de MPI en varios hosts, se deben seguir los pasos indicados en el Anexo del presente guión.

Anexo I: Ejecución en el Clúster

Disponemos de tres MVs con S.O. Linux y, por ejemplo, las siguientes IPs:

- Nodo 1: 192.168.102.126
- Nodo 2: 192.168.102.127
- Nodo 2: 192.168.102.128

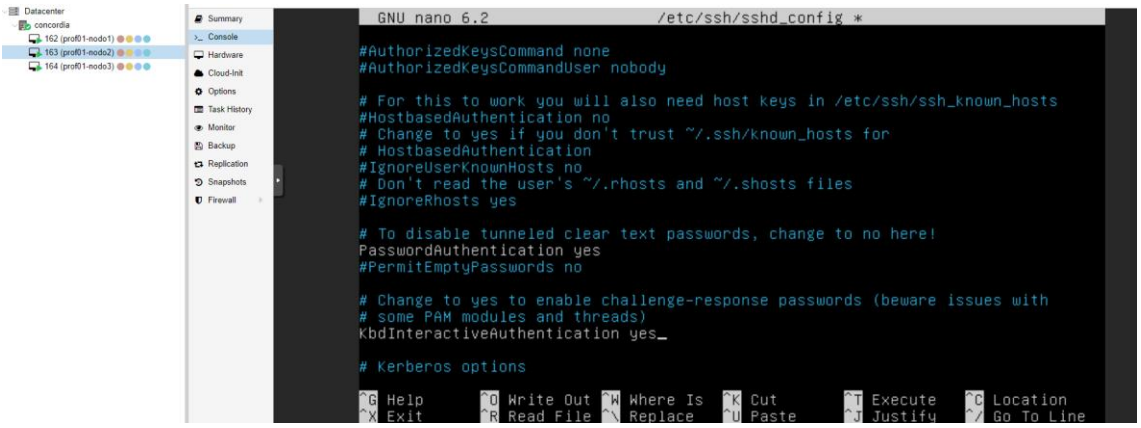
Se pueden consultar las características en la opción de “Summary”:



Los pasos a seguir para ejecutar un programa MPI en los tres nodos se detallan a continuación:

1. Modificar en cada uno de los nodos el fichero `/etc/ssh/sshd_config`:

`sudo nano /etc/ssh/sshd_config`:

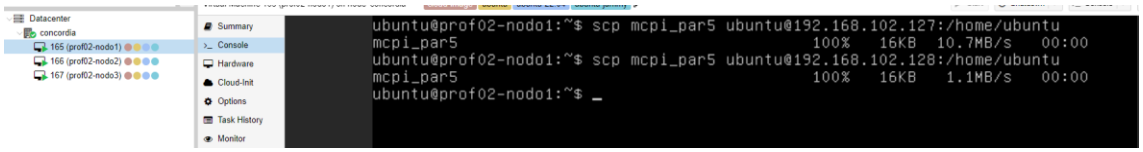


2. A continuación, reiniciar el servicio ssh:

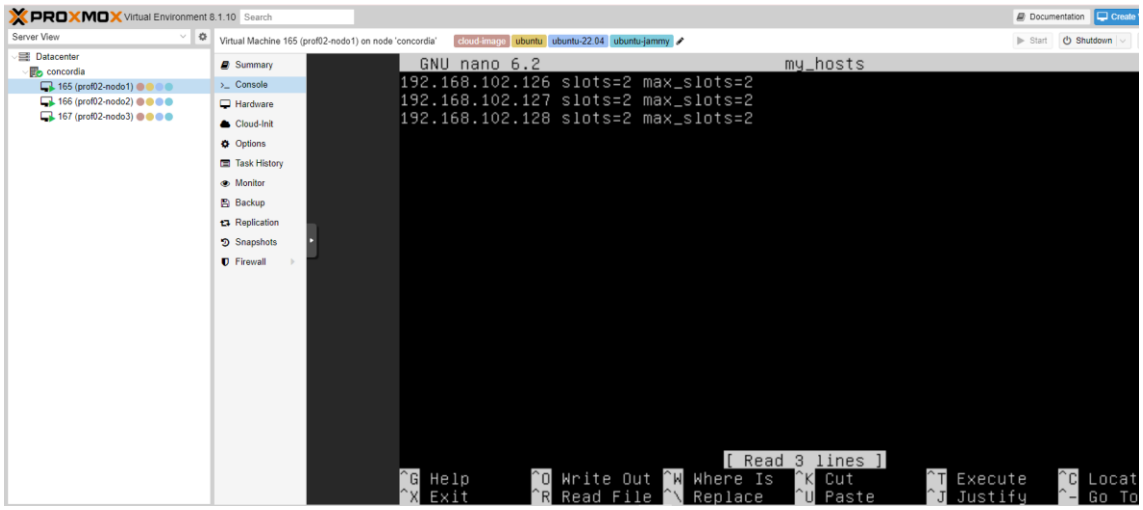
`sudo service ssh restart`

3. Copiamos el fichero a ejecutar en cada uno de los nodos mediante el comando scp:

`scp mcpi_par5 ubuntu@192.168.102.127:/home/ubuntu`



4. Creamos el fichero my_hosts: `nano my_hosts`



5. Ejecutamos mediante el comando mpirun:

```
mpirun -np 6 -hostfile my_hosts /home/ububtu/mcpi_par5
```